Evaluating 30-Day Hospital Readmission Prediction in Patients with Diabetes: Patient-Level Versus Encounter-Level Validation

Muhammad Shehzore Hashmi
Electrical and Computer Engineering
Montana State University
Bozeman, Montana, USA
muhammadshehz.hashmi@student.montana.edu

Ahmad Sultan
Gianforte School of Computing
Montana State University
Bozeman, Montana, USA
ahmad.sultan@student.montana.edu

Akshay Kumar Dosapati
Gianforte School of Computing
Montana State University
Bozeman, Montana, USA
akshay.dosapati@student.montana.edu

***Abstract*—Many patients in hospital readmission datasets have more than one encounter, so the way records are split between training and testing can affect reported performance. We study this effect on the Diabetes 130-US Hospitals dataset. After excluding death and hospice discharges, the cohort contains 99,343 encounters from 69,990 patients, of which 11.39% are readmitted within 30 days. We compare L2 and L1 logistic regression, random forest, and LightGBM under two evaluation designs: a patient-level split, in which no patient appears in both training and test data, and a conventional stratified encounter-level split. Each design is repeated with five predefined seeds, using five-fold cross-validation on the development data and fitting all preprocessing on training folds only. Average precision (AP) is the primary metric. Under patient-level evaluation, the four class-weighted models reach mean AP between 0.208 and 0.228, with LightGBM highest. In encounter-level test sets, 41.7% of encounters on average belong to patients who also appear in the development data. Encounter-level AP is slightly higher (+0.006 to +0.013) and the ranking of the models changes. These preliminary results suggest that the choice of split matters more for comparing models than for their overall performance level. Probability calibration, resampling, and subgroup analyses are still in progress.**

***Index Terms*—readmission, diabetes, grouped validation, repeated encounters, average precision**

I. Introduction

Readmission within 30 days increases healthcare costs and can point to problems with recovery, discharge planning, or follow-up care [1]. Patients with diabetes often have several diagnoses and repeated admissions, which makes their risk hard to assess. A model that flags high-risk encounters at discharge could help hospitals target follow-up resources, but only if its reported performance reflects how it would perform on new patients. We therefore study both the prediction task and how reliably it is evaluated.

The Diabetes 130-US Hospitals dataset contains inpatient encounters recorded between 1999 and 2008 [2], [3]. Each record is an encounter, not a patient, and many patients have several encounters. If encounters are split at random, the test set will contain both new patients and patients whose other visits were used in training. A patient-level split avoids this by keeping all of a patient's encounters on the same side. Removing the patient ID from the inputs does not solve the problem, because diagnoses, medications, and utilization history often repeat across visits. At the same time, overlap alone does not tell us how much it changes the results; that has to be measured.

Earlier work has studied clinical associations in this dataset [3], compared machine learning models [1], and adopted patient-level safeguards [4]. What is less well documented is how much the choice between patient-level and encounter-level evaluation changes the reported numbers and the ranking of models on the same cohort. We address three research questions. **RQ1**: How well do four established models rank readmitted encounters when evaluated on unseen patients? **RQ2**: How do average precision and the model ranking change under encounter-level evaluation, and are the changes consistent across seeds? **RQ3**: How much patient overlap does encounter-level splitting create, and how do calibration and performance on seen versus unseen patients help interpret the difference? Predictions are made at discharge, and our main target is generalization to unseen patients within this historical dataset.

This report makes three contributions: (1) an audit of the cohort and its repeated patients, which treats unperformed laboratory tests separately from unknown values; (2) a reproducible comparison of four model families under two validation designs and five seeds; and (3) an analysis of patient overlap, model ranking, and calibration that reports uncertainty alongside the point estimates. The target venue is the IEEE Intermountain Engineering, Technology, and Computing Conference.

II. Related Work

A. Clinical analysis and predictive benchmarking

Strack et al. [3] kept only the first encounter of each patient and excluded death and hospice outcomes. This sidesteps repeated observations by changing the cohort, which suits their analysis of HbA1c testing and readmission. We keep all eligible encounters because the repeated visits are what we want to study, so their sample sizes and associations are not directly comparable to our results.

Emi-Johnson and Nkrumah [1] compare logistic regression, random forest, XGBoost, and a neural network using a stratified 80/20 split, class weighting, cross-validation, and SHAP. Their methods section does not state that patients were kept apart between training and test data. This does not mean that their results are affected by leakage, only that the reported procedure does not let a reader rule it out. Their study motivates our model comparison, but we focus on how evaluation design changes results rather than on beating published scores.

B. Leakage safeguards and the closest comparison

Afridi et al. [4] describe patient-level partitioning, preprocessing and feature selection within folds, SMOTENC with class weighting, and SHAP/LIME explanations. Their cohort contains 75,081 encounters and they fix the random seed at 42. These safeguards already exist in their work. What we add is a repeated, side-by-side comparison of the two split designs, a measurement of patient overlap, and a check of whether the model ranking is stable across seeds (Table I).

C. Validation matched to the intended use

Saeb et al. [5] argue that validation must match how a clinical model will be used: when records from the same person appear in both training and test data, estimates for new patients become overly optimistic. Kapoor and Narayanan [6] place this problem within a broader taxonomy of data leakage in machine learning research. These studies motivate our choice of a patient-level split as the primary evaluation and our explicit overlap measurement. We apply their reasoning to repeated diabetes encounters and keep the encounter-level split as a comparison to measure how much the choice matters. Because the dataset has no usable dates, neither split can test prediction of a known patient's future visits.

D. Imbalance and evaluation

Precision–recall analysis is more informative than ROC analysis when positives are rare [7]. We summarize it with non-interpolated average precision rather than the trapezoidal area under the PR curve [8], and include a constant-score baseline whose AP equals the test prevalence. Scikit-learn's group-aware splitters implement the patient-level design [9]. Class weighting and resampling both change the meaning of predicted probabilities, so we report Brier scores and an unweighted logistic model alongside ranking metrics. A SMOTENC comparison using imbalanced-learn [10] is planned. LightGBM [11] provides an efficient boosted-tree model alongside random forest and regularized logistic regression.

III. Data and Preprocessing

A. Source, cohort, and outcome

We downloaded the public UCI files [2], including the mapping of categorical IDs, and recorded a SHA-256 checksum of the data file. The CSV has 101,766 rows and 50 columns: two identifiers, the outcome, and 47 candidate predictors. Each row is one inpatient encounter. The outcome is $y = 1$ for `<30` and

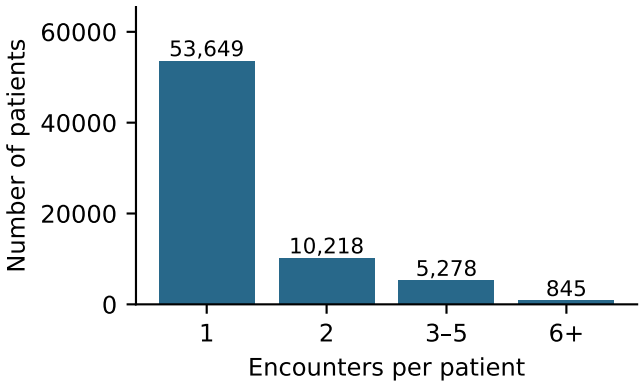


Fig. 1. Eligible patients grouped by number of encounters. Repeat encounters are kept and grouped by patient during the primary evaluation.

$y = 0$ for `>30` or `NO`; we did not derive any other readmission labels.

We exclude discharge codes 11, 13, 14, 19, 20, and 21, which denote death or hospice in the supplied mapping. This removes 2,423 rows and leaves 99,343 encounters from 69,990 patients, with 11,314 positives (11.39%). All age and gender categories are kept. Transfers remain in the cohort, with the discharge category used as a predictor; because a transfer is not an ordinary discharge home, we plan a sensitivity analysis without them.

B. Data quality audit

The data contain no exact duplicate rows, no repeated encounter IDs, and no rows that become duplicates when the two identifiers are removed. Repeated patients are therefore distinct visits, not duplicated records. In the eligible cohort, 16,341 patients (23.35%) have more than one encounter, and together they account for 45,694 encounters (46.00%). The median patient has one encounter, the 99th percentile six, and the maximum 40 (Fig. 1). For this reason we keep repeat visits and group them by patient during evaluation instead of deleting them.

Table II separates tests that were not performed from values that are unknown. In the HbA1c and glucose fields, `None` means the test was not performed [3], so we read it literally and encode it as a "Not performed" category rather than imputing a value. The `?` marker is encoded as a separate "Unknown" category. As planned in Report 1, weight is dropped because 96.85% of values are missing. The three diagnosis fields have 20, 356, and 1,419 unknown codes, which are also kept as an explicit category. We do not remove outliers or winsorize any variable.

C. Feature representation and leakage prevention

After dropping weight and the two IDs, 46 input variables remain. The eight count and duration variables are standardized, and all other variables, including age bands and admission and discharge codes, are one-hot encoded. ICD-9 diagnosis codes are grouped into nine clinical categories plus

TABLE I
COMPARISON OF RELEVANT STUDIES AND THIS REPORT. "NOT EXPLICIT" MEANS THE PUBLISHED METHODS DO NOT STATE IT, NOT THAT IT WAS DONE INCORRECTLY.

Study	Repeated patients	Validation	Leakage/imbalance handling	Relation to our study
Strack et al. [3]	First encounter per patient retained	Association modeling; no predictive holdout comparison	Death/hospice exclusions; clinical covariate analysis	Dataset definitions and cohort rationale
Emi-Johnson and Nkrumah [1]	Patient grouping not explicit in split description	Stratified 80/20; five-fold model tuning	Preprocessing pipeline and class weighting reported	Conventional predictive benchmark
Afridi et al. [4]	Patient groups kept separate	Patient holdout; five-fold grouped validation; seed 42	Preprocessing, feature selection and SMOTENC within training folds	Closest framework; safeguards already established
This report	All eligible encounters retained	Two split designs, five seeds; equal tuning budgets	All fitting on training data; overlap checks; no test resampling	Measures sensitivity to split design and ranking stability

TABLE II
AUDIT OF THE ELIGIBLE COHORT. UNKNOWN VALUES ARE COUNTED SEPARATELY FROM TESTS THAT WERE NOT PERFORMED.

Item	Count (percentage)
Eligible encounters / patients	99,343 / 69,990
Positive / negative outcomes	11,314 / 88,029
Patients with repeat encounters	16,341 (23.35%)
Encounters from repeat patients	45,694 (46.00%)
Unknown weight	96,218 (96.85%)
Unknown medical specialty	48,616 (48.94%)
Unknown payer code	39,398 (39.66%)
Unknown race	2,234 (2.25%)
HbA1c not performed	82,509 (83.05%)
Glucose serum test not performed	94,191 (94.81%)

"Unknown," following Strack et al. [3] but applied to all three diagnosis fields; supplementary E and V codes are assigned to "Other." The scaler and encoder are fitted on training folds only, and categories that appear only in held-out data are ignored. No numerical values are missing in this cohort.

Patient and encounter IDs are used only to form the splits and check them, never as predictors. Fixed cohort rules and code mappings are applied before splitting, and every learned step happens after it (Fig. 2). We do not build features from other encounters; the prior-visit counts are used as supplied. Because the data lack usable dates and hospital identifiers, our results do not speak to generalization over time or to new hospitals.

IV. METHODOLOGY AND IMPLEMENTATION

A. Models

The simplest baseline is a constant predictor that outputs the readmission rate of the development set. L2-regularized logistic regression is the main model baseline. L1-regularized (LASSO) logistic regression [12] drives some coefficients to exactly zero and so performs feature selection within the model. Both use class weights $w_k = n/(2n_k)$ computed from the training labels and minimize a regularized, weighted log loss. The inverse penalty strength C is chosen from $\{0.1, 1\}$ for L2 and $\{0.01, 0.1\}$ for L1, using the liblinear solver with up to 1,000 iterations. An otherwise identical L2 model without class weights serves as a reference for probability quality.

Random forest [13] uses 100 trees with maximum depth 12 and square-root feature sampling, with the minimum leaf size chosen from $\{5, 20\}$. LightGBM uses 150 boosting rounds, learning rate 0.05, at least 50 samples per leaf, and 15 or 31 leaves. Both use balanced class weights computed from the training data. We deliberately kept each search to two candidates, so that every model receives the same tuning budget and the full study fits in our compute limits; this is not an exhaustive benchmark.

B. Implementation status

The cohort audit, preprocessing pipeline, all five models, both split designs, repeated tuning and evaluation, and result generation are complete. L1 regularization is the only feature selection method implemented so far; we have not run recursive feature elimination. SMOTENC, probability recalibration, SHAP and permutation importance, and the discharge and subgroup analyses are planned, so the report makes no claims about them.

The experiments use Python 3.12, NumPy, pandas, SciPy, scikit-learn 1.8, LightGBM 4.7, and Matplotlib [14], [15], [16], [17], [11], [18], running on CPU only; exact versions are stored with the results. We met two practical problems. Random forest was slow on the sparse one-hot matrix, so we pass it a dense 32-bit matrix instead, which gives the same model much faster. A timing pilot also showed that the L1 models were slow at larger C values, so we narrowed the L1 grid before any test results were computed. The five seeds ran in parallel on an eight-core allocation with two threads per model. Each outer partition took 266–367 seconds (median 314), and the longest seed took about 12 minutes for both designs. Code, aggregate results, and the manuscript source are available at

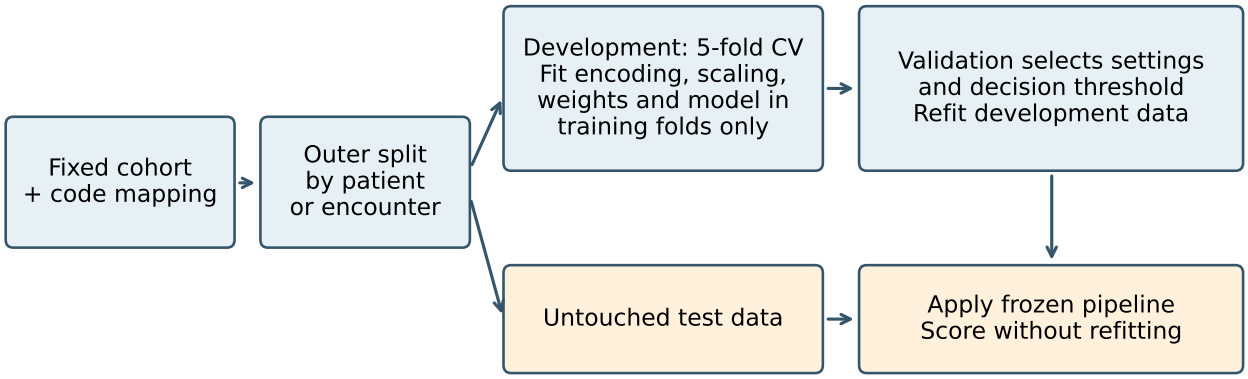


Fig. 2. Implemented workflow. The test data are never used for fitting or selection. L1 feature selection happens inside training folds. SMOTENC is planned and will also be restricted to training folds.

https://github.com/Ahmadjajja/Hospital_Readmission_Prediction.

V. Experimental Design

A. Data splitting and model selection

For each seed in $\{11, 23, 42, 67, 101\}$, the patient-level design holds out 20% of patients (GroupShuffleSplit), together with all of their encounters, so the test set holds about 20% of encounters. The encounter-level design holds out a stratified random 20% of encounters. Within the remaining development data, the patient-level design uses StratifiedGroupKFold and the encounter-level design uses StratifiedKFold, each with five folds. The code checks that no patient appears on both sides of any patient-level split.

Each candidate configuration is scored by its mean AP across the five validation folds, and the best one is refitted on the full development set. The decision threshold that maximizes F1 on the out-of-fold development predictions is then fixed and applied to the test set, which is used only once. Both designs use the same candidates and preprocessing, but each selects its own settings, so the comparison measures the effect of the whole evaluation procedure rather than overlap in isolation.

B. Metrics and uncertainty

Average precision is computed as

$$\mathrm{AP} = \sum_j (R_j - R_{j-1}) P_j, \tag{1}$$

where P_j and R_j are the precision and recall at the j-th threshold of the ranked scores [8]. We also report ROC-AUC, precision, recall, F1, and the Brier score $n^{-1} \sum_i (p_i - y_i)^2$. The threshold affects only precision, recall, and F1. We do not use accuracy, because a model that never predicts readmission would already score about 89%.

Across seeds we report means and sample standard deviations. The five splits share patients, so their spread shows how much results vary between partitions rather than providing a formal confidence interval. To compare models on the same patients, we use the seed-42 patient-level test set, chosen in advance, and draw 1,000 bootstrap samples of patients (resampling seed 2026). All encounters of a sampled patient are included together, and every model is evaluated on the same samples, giving paired percentile 95% intervals for the AP difference from L2. These intervals hold the fitted models fixed and are not adjusted for multiple comparisons. We do not compute paired intervals between the two designs, because their test sets differ.

C. Overlap analysis and success criteria

For each encounter-level test set, we record the share of test encounters whose patient also appears in the development data, and we report results separately for seen and unseen patients. We consider the study successful if it produces a complete, reproducible comparison with uncertainty estimates, even if the more complex models do not beat the baseline or the gap between designs turns out to be small.

VI. Implementation Progress and Initial Findings

A. Completed experiments and performance

All ten outer partitions (five seeds, two protocols) have been run, giving 60 evaluated model results. A verification script recomputes every reported metric from the saved predictions and checks labels and partitions against the source data. Table III and Fig. 3 summarize the results. These are preliminary findings on a historical benchmark.

All models perform well above the constant reference, but the absolute level is modest. LightGBM has the highest mean patient-level AP (0.228), compared with 0.216 for weighted L2 and 0.112 for the constant reference. Every model scores higher under encounter-level evaluation; the differences in mean AP are +0.0055 (L2 logistic), +0.0062 (L1 logistic), +0.0132 (Random forest), +0.0102 (LightGBM). Random forest shows the largest gap.

TABLE III
TEST PERFORMANCE, MEAN ± SAMPLE SD ACROSS FIVE SEEDS. CONSTANT AP EQUALS EACH TEST SET'S POSITIVE FRACTION.

Model	Split	AP	ROC-AUC
L2 logistic	Patient	0.216 ± 0.006	0.663 ± 0.003
L2 logistic	Encounter	0.221 ± 0.002	0.669 ± 0.005
L1 logistic	Patient	0.215 ± 0.006	0.665 ± 0.003
L1 logistic	Encounter	0.222 ± 0.002	0.671 ± 0.004
Random forest	Patient	0.208 ± 0.002	0.665 ± 0.005
Random forest	Encounter	0.221 ± 0.002	0.673 ± 0.004
LightGBM	Patient	0.228 ± 0.006	0.673 ± 0.002
LightGBM	Encounter	0.238 ± 0.004	0.679 ± 0.003
L2 unweighted	Patient	0.216 ± 0.006	0.662 ± 0.003
L2 unweighted	Encounter	0.221 ± 0.002	0.668 ± 0.005
Constant	Patient	0.112 ± 0.002	0.500 ± 0.000
Constant	Encounter	0.114 ± 0.000	0.500 ± 0.000

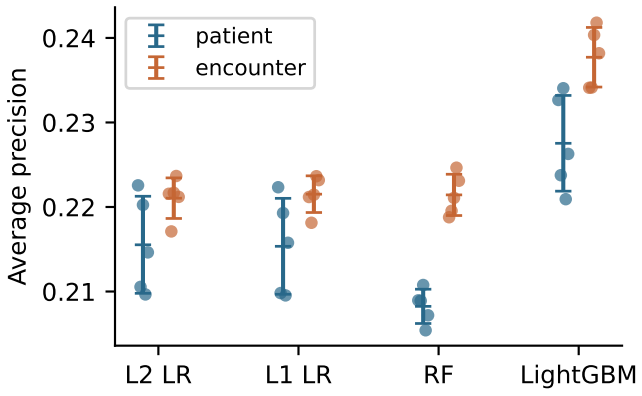


Fig. 3. AP across five seeds. Points are individual runs; marks and error bars show means and sample standard deviations.

B. Patient overlap and uncertainty

By construction, patient-level holdouts share no patients with the development data. In the encounter-level holdouts, 41.34% to 42.10% of test encounters (mean 41.70%) come from patients who also appear in development. Because the patient split holds out patients rather than encounters, its test sets vary slightly in size (19,717 to 19,991 encounters) and prevalence (10.99% to 11.41%), which the constant reference reflects.

Within the encounter-level holdouts, LightGBM reaches a mean AP of 0.279 on patients seen during development and 0.157 on unseen patients. The two groups also differ sharply in readmission rate (20.10% versus 5.16%). Seen patients are by definition repeat visitors, who are readmitted far more often, so this contrast mixes overlap with case mix.

For the seed-42 patient holdout, the paired AP differences relative to weighted L2, with 95% patient-bootstrap intervals, are L1 logistic: -0.0010 [-0.0026, +0.0006]; Random forest: -0.0095 [-0.0169, -0.0021]; LightGBM: +0.0138 [+0.0073, +0.0214].

TABLE IV
PATIENT-LEVEL THRESHOLD AND PROBABILITY DIAGNOSTICS: MEANS ACROSS FIVE RUNS. THRESHOLDS MAXIMIZE DEVELOPMENT OUT-OF-FOLD F1.

Model	Precision	Recall	F1	Brier
L2 logistic	0.193	0.475	0.274	0.225
L1 logistic	0.193	0.476	0.275	0.225
Random forest	0.189	0.500	0.274	0.219
LightGBM	0.211	0.435	0.284	0.215
L2 unweighted	0.197	0.447	0.272	0.095
Constant	0.000	0.000	0.000	0.099

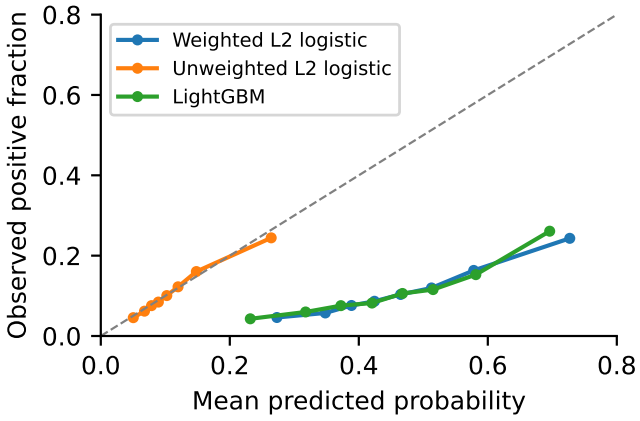


Fig. 4. Calibration curves for the seed-42 patient holdout, using eight quantile bins. The dashed line marks perfect calibration.

C. Thresholds and probability quality

Table IV shows the precision–recall trade-off at the thresholds chosen on development data. At these thresholds the models flag roughly half of the readmissions, and about one in five flagged encounters is a true readmission. Probability quality differs much more than ranking. The class-weighted models have Brier scores of about 0.22, worse than the constant reference (0.099), whereas unweighted L2 reaches 0.095 with almost the same AP. Fig. 4 shows why: class weighting pushes predicted probabilities far above the observed readmission rates. These scores therefore cannot be read as risks until they are recalibrated, which is planned.

D. Summary of initial findings

Report 1 defined a clear improvement over the baseline as a paired 95% interval that excludes zero. On the seed-42 holdout, LightGBM meets this criterion, random forest is significantly worse, and L1 logistic regression is indistinguishable from L2. Across seeds, the gap between the two protocols is small (+0.006 to +0.013 AP), but it is large enough to change conclusions about model choice: random forest ranks last under patient evaluation (0.208 AP) yet ties L2 under encounter evaluation (both 0.221).

VII. DISCUSSION, LIMITATIONS, AND RESPONSIBLE USE

The two designs answer different questions. A patient-level split estimates performance on new patients, while an

encounter-level split mixes new and returning patients. Our results show that the encounter-level design gives only slightly higher scores, but the gain is uneven across models and is enough to change which model looks best. Because the two designs use different test sets, the gap cannot be attributed to overlap alone.

Our cohort differs from those of earlier studies, mainly because we keep repeat encounters and transfers, so comparing our scores directly with published numbers would be misleading. The small tuning grids may favor some models over others. Encounter-level metrics also give more weight to frequent patients; we plan a patient-weighted analysis to check this. Readmissions to hospitals outside the network are not recorded, and clinical practice and coding have changed since 1999–2008.

The dataset is released under CC BY 4.0 [2]. It is deidentified, but demographic fields remain sensitive, and we make no attempt at reidentification. Race, gender, and age groups may be unevenly represented or recorded, and we have not yet analyzed performance by subgroup, so we make no fairness claims. The models are research benchmarks, not clinical tools, and any feature importance we report later will describe associations, not causes.

VIII. Response to Feedback and Updated Outline

We addressed each comment on Progress Report 1 as follows.

- *Figure size and readability:* the workflow figure (Fig. 1 in Report 1) is now Fig. 2, redrawn at full page width with larger labels.
- *Order of splitting, feature selection, and resampling:* the new figure shows the split first. Preprocessing and L1 feature selection are fitted only within training folds, and planned resampling will be restricted to training folds as well.
- *Repeated seeds:* every comparison is repeated with five predefined seeds, and we report means, standard deviations, and individual runs (Table III, Fig. 3).
- *Literature and Afridi et al.:* Table I compares how prior studies handle repeated patients, validation, and leakage. Section II-B explains what our study adds beyond Afridi et al.
- *Duplicates, missing data, status, and references:* Section III-B reports the duplicate checks and the encounters-per-patient distribution, and Table II separates unperformed tests from unknown values. Completed and planned work are labeled throughout, and all references follow IEEE style.

The research questions, dataset, and models are unchanged; we added an unweighted logistic model to study calibration. Table V gives the updated outline.

IX. Remaining Work and Responsibilities

Table VI lists the work remaining before Progress Report 3. Ahmad Sultan leads implementation and reproducibility, Muhammad Shehzore Hashmi leads validation and statistical checks, and Akshay Kumar Dosapati leads the literature review and figures. All three members review the code, the cited papers, the results, and the manuscript. The project is on the schedule proposed in Report 1: the primary evaluation was planned for early November, and the imbalance, calibration, and sensitivity analyses for mid-November. The scope has not changed. If time runs short, we will run resampling on one model only and report subgroup counts without comparisons where groups are too small.

TABLE V
Updated outline of the final paper.

Section	Content, evidence, and planned outputs
Abstract	Update findings after the remaining analyses.
Introduction	Keep RQs and contributions; adjust scope to the final evidence.
Related work	Add calibration and resampling literature; extend Table I.
Data	Keep the audit table and encounter-frequency figure.
Methods	Add SMOTENC and calibration settings.
Experiments	Same seeds and grouping; add sensitivity and subgroup protocols.
Results	Current AP/ROC tables plus an imbalance table, a subgroup table, and a feature-importance figure.
Discussion	Interpret ranking changes, uncertainty, and calibration across designs.
Limitations	Historical data, repeated patients, no dates, representation, external validity.
Conclusion	Answer each RQ with the completed evidence.

TABLE VI
Remaining work toward Progress Report 3, in priority order.

Target	Task and dependency	Lead
Week 1	Review literature on patient- versus record-level validation and calibration; extend Table I	Shehzore, all
Week 1–2	SMOTENC within training folds on a limited model set; needs a mixed-type pipeline	Ahmad
Week 2	Recalibration fitted on development data; transfer and patient-weighted sensitivity analyses	Ahmad, Shehzore
Week 2–3	Subgroup counts and errors; feature-importance figure; needs enough positives per group	Akshay, all
Before R3	Add imbalance and subgroup tables and calibration and importance figures; final review	All members

X. Preliminary Conclusion

Repeated patients account for nearly half of all encounters. Evaluating on encounters instead of patients raises AP only slightly but changes how the models compare. LightGBM performs best on unseen patients, and the class-weighted models need recalibration before their outputs can be read as risks.

References

[1] O. G. Emi-Johnson and K. J. Nkrumah, "Predicting 30-day hospital readmission in patients with diabetes using machine learning on electronic health record data," *Cureus*, vol. 17, no. 4, p. e82437, 2025.

[2] J. Clore, K. Cios, J. DeShazo, and B. Strack, "Diabetes 130-US Hospitals for Years 1999–2008," UCI Machine Learning Repository, 2014, dataset, CC BY 4.0. [Online]. Available: https://doi.org/10.24432/C5230J

[3] B. Strack, J. P. DeShazo, C. Gennings, J. L. Olmo, S. Ventura, K. J. Cios, and J. N. Clore, "Impact of HbA1c measurement on hospital readmission rates: Analysis of 70,000 clinical database patient records," *BioMed Research International*, vol. 2014, p. 781670, 2014.

[4] A. A. Afridi, M. Usman, D. J. Rubin, and M. R. R. Rana, "A robust and explainable framework for predicting hospital readmission in patients with diabetes," *Diabetology*, vol. 7, no. 10, p. 197, Oct. 2026.

[5] S. Saeb, L. Lonini, A. Jayaraman, D. C. Mohr, and K. P. Kording, "The need to approximate the use-case in clinical machine learning," *GigaScience*, vol. 6, no. 5, p. gix019, 2017.

[6] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, p. 100804, 2023.

[7] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLOS ONE*, vol. 10, no. 3, p. e0118432, 2015.

[8] scikit-learn developers, "average_precision_score," scikit-learn documentation, accessed: Nov. 2, 2026. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.average_precision_score.html

[9] ——, "Cross-validation: Evaluating estimator performance," scikit-learn documentation, accessed: Nov. 2, 2026. [Online]. Available: https://scikit-learn.org/stable/modules/cross_validation.html

[10] G. Lemaître, F. Nogueira, and C. K. Aridas, "Imbalanced-learn: A python toolbox to tackle the curse of imbalanced datasets in machine learning," *Journal of Machine Learning Research*, vol. 18, no. 17, pp. 1–5, 2017.

[11] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 3146–3154.

[12] R. Tibshirani, "Regression shrinkage and selection via the lasso," *Journal of the Royal Statistical Society: Series B*, vol. 58, no. 1, pp. 267–288, 1996.

[13] L. Breiman, "Random forests," *Machine Learning*, vol. 45, pp. 5–32, 2001.

[14] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

[15] W. McKinney, "Data structures for statistical computing in Python," in *Proceedings of the 9th Python in Science Conference*, 2010, pp. 56–61.

[16] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[17] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[18] J. D. Hunter, "Matplotlib: A 2d graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.